

ENIGMA V0.5.3.7.4 — Реестр багов и вердикт о шансах

Сводный аудит на основе исходного кода, документации (docs/ , docs/Почти Актуальные TZ/), логов (backend/logs/ , cds_session_*.log , causal_validation.log , sleep_test2.log , scene_changes.jsonl), тестового набора IPT.py и интеграционных прогонов SUPERBOX.

Источники для каждого бага приведены в формате file:line или log:file . Вердикты по приоритетам — мои собственные, на основе частоты в проде и игровой критичности.

Часть 0 — Ответ на главный вопрос: шансы, что «всё получится»

Что имеется в виду под «эмерджентной драмой»

Эмерджентная драма в понимании автора — это не «скриптованные диалоги», а **драма, возникающая из каузальной симуляции**: NPC имеют внутреннюю жизнь (L1Chronicle, CrystallizedBelief, MemoryManager), взаимодействуют друг с другом (NPC ↔ NPC dialogue, SocialPhysics, RelationshipStore), реагируют на игрока (DM-agent), а мир накапливает историю поколений (WorldChronicle, lineage_time). Игрок не «проходит квесты» — он попадает в чужую живую систему со своими правилами.

Текущее состояние (V0.5.3.7.4)

На уровне архитектуры проект уникален. Я не нашёл аналогов в публичном поле, которые бы одновременно:

- Хранили identity как append-only chronicle (L1), а state как ephemeral snapshot
- Имели формализованные эпистемические границы (NPC не читают мысли друг друга, ADR-0-322)

- Запрещали `random.*` и `time.time()` в ядре симуляции (ADR-O-301, §15)
- Имели 12 кастомных архитектурных линтеров
(`lint_silent_failures.py`, `lint_hp_ssot.py`, `lint_kernel_rng.py`,
`lint_wall_clock.py`, `lint_epistemic_boundary.py`,
`lint_l1_append_only.py`, `lint_spatial_ssot.py`,
`lint_position_mutation.py`, `lint_frontend_isolation.py`,
`lint_domain_purity.py`, `lint_llm_exile.py`,
`lint_perception_architecture.py`)
- Использовали Dual-Rail equivalence validator с drift-классификацией (ADR-O-201)
- Формализовали контракты через 111 ADR + IMPACT-файлы

На уровне исполнения проект болен. Из логов:

- 22/40 тиков `sleep_test` падают с `'str' object has no attribute 'target_id'` — Phase 5 decision ломается на каждом тике после 18-го
- 3/6 NPC спят не в своих кроватях (`exit_west` вместо `tent_*` / `guard_bed_*`)
- DM на 23 из 25 тестовых промптов отвечает «Ничего не произошло» (включая атаковать трактирщика — HP остаётся 100)
- 553 рестарта llama-server за сессию (crash-restart churn)
- 807 RNA_WORKER "Все модели пула недоступны" за сессию
- `identity_integrity` падает 0.93 → 0.00 после одной атаки + 100 idle тиков (нарушает дизайн-принцип «Нет злодеев»)
- Movement Traversal ❌ для всех 6 NPC в последней зафиксированной сессии (reports/LAST_SESSION.md, 2026-08-06)
- `scene_changes.jsonl` (379 579 строк): 12 truncated JSON строк (save corruption), 84.7% записей с пустым `target_location_id`
- `backend/logs/error.log: /api/game/action` падает с HTTP 500 из-за `TickBuffer.__init__() got an unexpected keyword argument 'mvp_controller'` — **система self-healing крашит пайплайн, который должна мониторить**
- 1 131 INV-TEMPORAL-ISOLATION нарушений (TickState мутируется во время пайплайна — race condition)

Мой честный вердикт о шансах

Сценарий	Шанс	Почему
А. Никогда не выйдет как игра, останется «исследовательским артефактом»	≈ 55–65%	Архитектура углубляется (Epochs 7–10) быстрее, чем стабилизируется базовая механика. Уже сейчас 5 эпох пройдено, но базовое Movement Traversal не работает. Solo-разработчик с AI-ассистентом не вытянет 11 эпох.
В. Выйдет как нишевый RU-релиз через 1.5–2 года дисциплинированной работы	≈ 25–30%	Если заморозить горизонтальное расширение (Epochs 8–10 отложить) и сделать 2–3 цикла чистой стабилизации — реально. Аудитория: фанаты Disco Elysium / Weyrwood / Cataclysm: DDA. Достижимая ниша.
С. Технологическая демка / статья на GDC / open-source фреймворк	≈ 10–15%	Архитектура (SSOT, Causal Contract, ADR-Net, CDS, IPT, PBT) — это материал для технической статьи или framework. Если переупаковать — есть шанс стать «Dwarf Fortress эпистемической симуляции».
Д. Мейнстримный коммерческий успех	< 5%	Rugame + локальная Llama 7B + 13k LOC god-методов + 288 silent excerpts — не мейнстрим. Это никогда не будет «обычной игрой». И это нормально.

Что нужно, чтобы случилось В (минимальные условия):

1. Заморозить Epochs 7–10. Никакого Prophecy / Generations / §18 до стабилизации Epoch 6.
2. Пройти **Phase 0 стабилизации** из ENIGMA_ROADMAP.md — закрыть P0-блокеры (см. ниже).
3. Ввести **недельный.freeze цикл**: каждую неделю — pull request с «удалением мёртвого кода», а не добавлением фич.

4. Иметь **green CI** на **6/6 NPC Movement Traversal** как обязательный gate.
5. Не использовать AI-ассистента для рефакторинга — только для генерации тестов. Рефакторинг — человеческими руками, с пониманием.

Главный риск: автор — интеллигентный визионер, который любит архитектуру больше, чем продукт. Это видно по соотношению `docs/` (320 .md) к тестам покраски игры (1 канарейка `test_full_playthrough_end_screen_non_empty`). Архитектура развита, игра — нет.

Главная надежда: архитектурный устав реально исполняемый. 12 кастомных линтеров работают. ADR-граф связный. Causal Contract v2.0 формализован. Это не «промышленный код», но и не «любительский» — это **академически серьёзный хобби-проект**, у которого есть шанс стать нишевым релизом, если автор сменит приоритет с «углубления модели» на «закрытие базовых дыр».

Часть 1 — КРИТИЧЕСКИЕ БАГИ (P0, блокируют запуск / MVP)

Эти баги **нельзя выпускать в любой релиз**. Они либо крашат игру на старте, либо ломают базовую механику (движение, бой, диалог, сохранения).

P0-1. `/api/game/action` возвращает HTTP 500 — игра не запускается

Источники: `backend/logs/error.log` (5 трейсбеков), `launcher_crash.log`, `game_loop/__init__.py:1227, 1551`

Симптомы:

```
AttributeError: 'PipelineContext' object has no attribute 'world_snapshot'
TypeError: TickBuffer.__init__() got an unexpected keyword argument 'mvp_controller'
```

`launcher_crash.log` показывает 4× HTTP 500 подряд при ожидании готовности backend. Лаунчер не может пройти стартовое меню «New Game».

Лечение:

1. `game_loop/__init__.py:1551` — `_ctx = TickContext(mvp_controller=self.mvp_controller)` передаёт `kwargs`, которого нет в сигнатуре `TickBuffer.__init__`. Или добавить `mvp_controller: Optional["MvpTavernController"] = None` в `TickBuffer.__init__`, или убрать `kwargs` и пробрасывать `mvp_controller` через `ctx.mvp_controller` атрибут после создания.
2. `game_loop/__init__.py:1227` — `state.shared_context.world_snapshot` обращается к атрибуту, которого нет на `PipelineContext`. Добавить `world_snapshot: Optional[WorldSnapshotDTO] = None` в `PipelineContext` и заполнять его в `_run_pipeline` после Phase 9.
3. После фикса — добавить канарей-тест `test_game_action_endpoint_returns_200` в `backend/tests/test_startup_checks.py`.

P0-2. `TickBuffer` self-healing probe крашит пайплайн

Источник: `game_loop/__init__.py:1551`, `tick_orchestrator.py` (`probe_runner wiring`)

Противоречие: `# ENIGMA SELF-HEALING: For probes` комментарий указывает, что `mvp_controller` передаётся для зондов — но `TickBuffer` (родительский класс) об этом не знает. Self-healing ломает то, что лечит.

Лечение: вынести `probe-wiring` в отдельный декоратор `@probes.observe("tick")` на `_run_pipeline`, а не тащить `probe-зависимости` в конструктор `TickBuffer`. `TickBuffer` должен быть чистым контейнером данных.

P0-3. Movement Traversal не работает ни для одного из 6 NPC

Источники: `reports/LAST_SESSION.md` (2026-08-06), `sleep_test2.log`, ADR-O-201/204/341

Все 6 NPC в последней зафиксированной сессии используют legacy fallback [TRAV_EXEC] Legacy traversal for {npc}, synthesizing WALK (15 804 срабатывания за сессию). А* pathfinding не вызывается, корректный traversal FSM (transition_traversal()) имеет **0 call sites** — все статусы ("MOVING" , "COMPLETED") пишутся напрямую в apply_change .

Лечение (по этапам):

1. Привести transition_traversal() к callable — добавить вызовы в scene_state_manager.apply_change (NPC_POSITION branch) и event_compiler._resolve_traversal_status .
2. Удалить legacy synthesizer в traversal_execution_system.py:resolve() — пусть кидает SimulationIntegrityError вместо synthesizing WALK . Сначала будут красные тесты, потом их починят.
3. После этого — INV-TRAV-TERMINALITY (IPT) пройдёт честно, а не через mock.

P0-4. social_input_ema — Save/Load игрока сломан (БАГ-3 из аудита v0.5.3.7.2)

Источник: player_avatar_service.py:300 , тест test_player_body_state_survives_save_load

TypeError: 'float' object is not iterable при dict(state.social_input_ema) . Поле имеет тип float в рантайме, но сериализация ожидает dict . Любой save/load игрока падает.

Лечение:

1. В models/player_avatar.py (или где определён PlayerAvatar) проверить тип social_input_ema: Dict[str, float] — должно быть dict, не float.
2. Если кто-то пишет туда float — найти writer (grep -rn "social_input_ema" backend/app) и поправить.
3. Добавить round-trip тест: state_in == state_out после save → load.

P0-5. topology_gate пропускает двери без wall_id

Источник: `tests/sandbox/movement/test_topology_gate.py`,
`graph_compiler.py`

`SimulationIntegrityError` не поднимается для карт, где дверь не привязана к стене. NPC и игрок могут проходить сквозь неразрезанные стены. Это объясняет 1 529 `INV-TOPOLOGY-WALL-CROSS` в логах — `tavern.json` имеет рёбра `entrance→kitchen`, пересекающие стену, и они молча удаляются (BUG-TOPO-001 FIX: Временно логируем и удаляем ребро, не роняя `SpatialService`).

Лечение:

1. В `graph_compiler._validate_connectivity` (или `topology_gate`) — поднимать `SimulationIntegrityError` если у двери нет `wall_id`.
2. В `tavern.json` — добавить `wall_id` всем door connections (или перерисовать рёбра вокруг стен).
3. Удалить soft-fallback в `graph_compiler.py:447` — заменить на crash с понятным сообщением.

P0-6. `BREAK_DELTA *` коллапс identity за один бой (дизайн-P0)

Источник: `causal_validation.log`, НАХОДКА-5 из аудита v0.5.3.7.2

`identity_integrity: 0.93 → 0.90 → 0.00` после одной атаки + 100 idle тиков. Прямое нарушение дизайн-принципа «Нет злодеев» (NPC ломаются от накопленных обстоятельств, не от одного удара). S157 MUTATIONS.md «P1: Balance Fix» недостаточен — коллапс происходит за один боевой контакт, не за «3 дня фонового стресса».

Лечение:

1. В `core/constants.py` — снизить `BREAK_DELTA *` ещё в 3–5 раз от текущих значений (уже снижались в 5 раз, но недостаточно).
2. Ввести асимптотику: `integrity_after = integrity * (1 - delta * integrity)` — чем ниже integrity, тем медленнее падает. Это даст «упругий» коллапс вместо линейного.
3. Тест: `test_break_progress_does_not_collapse_in_one_fight` — 5 атак подряд не должны опустить integrity ниже 0.5.
4. Тест: `test_break_progress_recovers_in_peace` — 100 idle тиков после атаки должны поднять integrity минимум на 0.1.

P0-7. Phase 5 decision крашит на каждом тике после 18-го

Источник: `sleep_test2.log` (22 TICK_CRASH), `phases/decision.py:206`

```
[TICK_CRASH] campaign=Open_road tick=19 error='str' object has
no attribute 'target_id'
File "...phases\decision.py", line 206, in ...
    if _r.target_id == "player":
```

`_r` — это `str`, а ожидается relationship DTO. BREAK_PROGRESS для `maid_lusya` сломан каждый тик.

Лечение:

- В `phases/decision.py:200-210` — проверить, что возвращает `relationships.get_pair(...)`. Если это `dict` (legacy), обернуть в DTO или использовать `rel.get("target_id")`.
- Добавить assertion в `RelationshipStore.get_pair`: возвращать `RelationshipDTO`, не `str`.
- Тест `test_phase5_decision_does_not_crash_on_str_relationship` — подать `str` и проверить graceful degradation.

P0-8. NPC sleep routing: 3/6 идут не в свои кровати

Источник: `sleep_test2.log:2893-2898`, `domain_spatial.md` BUG-SPATIAL-001

- `guard_borko`: ожидаемо `city_gate/guard_bed*`, реально `city_gate/exit_west` ❌
- `blacksmith_orm`: ожидаемо `city_gate/tent_*`, реально `city_gate/exit_west` ❌
- `merchant_goran`: ожидаемо `city_gate/tent_*`, реально `city_gate/exit_west` ❌

Финальный вердикт теста: 🌟 ТЕСТ СНА ПРОВАЛЕН!

Корневая причина: `movement_engine.py:323` мутирует `intent.target_node_id` in-place (был `"guard_bed"`, стал `"exit_west"`) при cross-loc материализации. Затем `materialize lookup` использует уже перезаписанную строку.

Лечение:

1. В `movement_engine.py` `cross-loc materialize` — не мутировать `intent.target_node_id`. Создать `ResolvedSpatialTarget` (ADR-O-330) с оригинальной целью, а `boundary node` держать в отдельном поле `boundary_node_id`.
2. После достижения `boundary node` — генерировать новый `MovementIntent` с оригинальным `target_node_id` в новой локации.
3. Удалить `[CROSS_LOC_INTERCEPT]` `reroute` в `exit_west` — заменить на `proper boundary traversal` с FSM.

P0-9. DM отвечает «Ничего не произошло» на всё (включая бой)

Источник: `causal_validation.log` (23× «Ничего не произошло»),
`domain_dialogue.md` Bug 1/2/3

Игрок: атаковать трактирщика → DM: Ничего не произошло. →
`tavern_keeper_tornin`: HP=100 (не уменьшился).

Каскад из трёх багов:

- **BUG-DLG-002** (`dm_agent.py`:245-249): `ValueError` когда `_has_target=False AND _has_stm=False AND _is_intro=False`.
`run()` проглатывает, возвращает `_fallback_narrate()` → `MSG_NOTHING_HAPPENED`.
- **BUG-DLG-006** (`game_loop/__init__.py`:1051):
`shared_context.all_npcs_raw_snapshot` никогда не присваивается.
LLM получает пустой контекст NPC.
- **BUG-DLG-018** (`response_validator.py`:74-75, 270-272):
`ResponseValidator` возвращает "Ничего не произошло." при любом нарушении (пустой ответ, CJK, 4th-wall слова).

Лечение:

1. `dm_agent.py`:245-249 — убрать `ValueError`. Если нет `target` и нет STM, всё равно строить `contract` с `empty target_block` и пометкой `is_ambient=True`. LLM должен уметь генерировать `ambient narration`.
2. `game_loop/__init__.py`:1051 — после `_run_pipeline` присвоить `shared_context.all_npcs_raw_snapshot = self._collect_npcs_raw(...)`.

3. `response_validator.py:270-272` — заменить «Ничего не произошло» на конкретные сообщения для каждого класса ошибки: `"[DM_NARRATION_REJECTED: empty]"`, `"[DM_NARRATION_REJECTED: 4th_wall]"` и т.д. Игрок не должен видеть «Ничего не произошло» — это UX-фальшивка.

P0-10. `player_threatens` не доходит до `CombatSubscriber`

Источник: `domain_dialogue.md` BUG-DLG-019/020, `phase_1_input.py:276-283`, `reaction_subscriber.py:52-65`

`_evt_map` в `phase_1_input.py` не содержит `player_threatens`, `player_steals`, `player_insults`, `player_flees` — все они падают в `PLAYER_SPOKE`. При этом `ReactionSubscriber._REACTION_EVENT_TYPES` не содержит `PLAYER_SPOKE`. Цепочка: угроза → `PLAYER_SPOKE` → игнор → ничего не происходит.

Лечение:

1. `phase_1_input.py:276-283` — добавить в `_evt_map`:

```
"player_threatens": EventType.PLAYER_THREATENS,
"player_steals": EventType.PLAYER_STEALS,
"player_insults": EventType.PLAYER_INSULTS,
"player_flees": EventType.PLAYER_FLEES,
```

2. `reaction_subscriber.py:52-65` — добавить `PLAYER_SPOKE` в `_REACTION_EVENT_TYPES` (или убрать его, если он не должен инициировать реакции — тогда `_evt_map` должен мапить на конкретные типы).
3. Тест: `test_player_threat_reaches_combat_subscriber`.

P0-11. BUG-CORE-003: `hub_event` не доходит до `TickState`

Источник: `domain_core.md` BUG-CORE-003, `pipeline_runner.py:60`, `dto.py:78-189`, `npc_orchestration.py:149-160`

`build_tick_state` ищет ключ `"dm_ctx"` в `intervention.payload`, но `npc_orchestration.py:149-160` строит `payload` с ключами `text`, `player_name`, `semantic_action`, `target_id`, `target_reference`, `tick` —

без `dm_ctx`. В итоге `_dm_ctx` всегда `None` → `state.hub_event=None` → `state.player_target_id=None` → `state.action_type="idle"`.

Это **корневая причина** «DM: Ничего не произошло». S177 помечен как partial fix (`task_scheduler` call возвращён), но **полный bridge от GameLoop до _TickContext не построен**.

Лечение:

1. В `npc_orchestration.py:149-160` — добавить `dm_ctx` в payload (или передавать его как отдельный аргумент `build_tick_state`).
2. В `pipeline_runner.build_tick_state` — добавить параметр `hub_event: Optional[InterventionEvent] = None`.
3. В `_TickContext` — добавить поле `hub_event: Optional[InterventionEvent] = None`.
4. Тест `test_hub_event_reaches_tick_state` — игрок атакует → `state.hub_event` не `None` → NPC реагирует.

P0-12. LLM crash-restart churn (553 рестарта за сессию)

Источник: `llama_server_stderr.log` (17.7 MB, 229к строк), `enigma_20260809.jsonl` (20× `startup_complete`)

Qwen2.5-7B перезагружается 553 раза. 400× `GET / ::1 503` (модель не готова). 217× `POST /abort ::1 404` (endpoint не существует в `llama.cpp`). 4× `POST /v1/chat/completions ::1 500` с CJK-мусором в prompt: 半空中飞过的雁 и 又要多少水 — это **mixed Cyrillic + CJK corruption** в теле запроса.

Лечение:

1. **/abort endpoint:** `llama.cpp` его не реализует. Использовать `httpx.AsyncClient` с cancellation token (`asyncio.CancelledError`) вместо HTTP abort.
2. **CJK corruption:** проверить кодировку при сериализации prompt в `llama_cpp_provider.py`. Скорее всего, `json.dumps(prompt, ensure_ascii=False)` + `subprocess.Popen` с `text=True` на Windows теряет байты. Заменить на `text=False, encoding='utf-8'` или передавать через stdin binary.
3. **Crash-restart loop:** в `llm/server_lifecycle.py` добавить exponential backoff между рестартами (1s → 2s → 4s → 8s → max 30s). Сейчас при падении сразу поднимается заново, не успевая отпустить VRAM.

4. **VRAM monitor:** 60× 'VramMonitor' object has no attribute 'start_session' + vram_after_mb: 0 во всех load_success. Метод не существует. Реализовать или убрать монитор.

P0-13. target_location_id пустой в 84.7% записей

scene_changes.jsonl

Источник: backend/data/logs/scene_changes.jsonl (379 579 строк)

321 260 из 379 579 записей имеют пустой target_location_id. Это upstream cause всех «NPC location mismatch» probe failures.

Лечение:

1. Найти writer: `grep -rn "target_location_id"`
backend/app/services/events/ backend/app/services/state/ .
 2. В EventDTO schema (или SceneChange DTO) — сделать target_location_id: str обязательным (не Optional).
 3. Везде, где создаётся SceneChange с NPC_POSITION, — извлекать target_location_id из npc_state.location_id.
 4. После фикса — INV-SCENE-ENTITY-ISOLATION начнёт проходить.
-

P0-14. scene_changes.jsonl — 12 truncated JSON (save corruption)

Источник: backend/data/logs/scene_changes.jsonl, строки 178 166, 348 456, 353 487, 353 773, 356 595–360 711

```
ion_id": "tavern"}
"target_location_id": ""}
""}
```

Кластер из 8 corruptions между L356595–L360711 — process crash mid-write.

Лечение:

1. В scene_state_manager._log_change — использовать atomic write pattern: `tempfile.NamedTemporaryFile(delete=False, dir=...) +`

`os.replace(temp, target)` . Или писать через SQLite WAL (с已经有 `SqliteMemoryStore` , использовать его).

2. Или: вообще убрать `_log_change` в JSONL — дублирует `scene_state_manager.apply_change` + CDS. Перенести в `L1Chronicle` (он уже append-only SQLite).
3. Добавить `fsync` после каждого `write` (или `buffer` + `flush` раз в N секунд).

P0-15. INV-TEMPORAL-ISOLATION — 1 131 нарушение (race condition)

Источник: `cds_session*.log`, INV-TEMPORAL-ISOLATION: Tick N: TickState mutated during pipeline execution (hash mismatch)

`StateApplicator` мутит `TickState` внутри `NpcTickPipeline.run()` (S179 частично фиксит, но нарушения продолжаются).

Лечение:

1. Проверить, что S179 (ADR-0-346) действительно убрал `StateApplicator.apply()` из `NpcTickPipeline.run` . Если да — найти **другой** мутатор (`grep state\.` inside `NpcTickPipeline.run`).
2. В `_assert_tick_invariants` — кидать `SimulationIntegrityError` (а не `warning`) при `hash mismatch`. Сейчас 1 131 violation — это значит, что `invariant degraded` до `noise level`.
3. Если мутации приходят из `async task scheduler` — добавить `asyncio.Lock` вокруг `TickState` access.

Часть 2 — ВЫСОКИЕ БАГИ (P1, ломают геймплей, но не критично для запуска)

P1-1. HP Double Truth (4 writer'a `target["hp"]` напрямую)

Источники: `combat_math.py:12,50,52,61` (fallback to global `random`), `combat_service.py:111` , `condition_engine.py` (HP writes), `apply_healing` (resurrects dead)

`body_state["current_hp"]` объявлен SSOT (ADR-HP-UNIFICATION), но 4 файла пишут в `state.hp` напрямую. Плюс `apply_healing` может

воскресить мёртвого NPC.

Лечение:

1. Везде заменить `target["hp"] = X` на `target["body_state"]`
`["current_hp"] = X`.
2. В `apply_healing` — early return если `life_status == DEAD`.
3. В `combat_math.py` — убрать fallback на `random` если `rng=None`;
кидать `ValueError("rng is required")`.
4. `lint_hp_sshot.py` — расширить, чтобы ловил `target["hp"]` и
`state.hp` (а не только `state.hp =`).

P1-2. 6 новых `random.*` violations (BUG-CORE-021..026)

Источники: `impact_engine.py:131` (`random.Random(hash(uuid))`),
`market_state.py`, `traveller.py`, `npc_conversation.py`,
`dm_response_normalizer.py:71` (`random.choice`), `attention_layer.py`,
`rules_agent.py` (`random.randint(1, 20)`), `llama_cpp_provider.py:163`
(`random.randint(0, 2**31-1)`)

ADR-O-301 требует `KernelRNG(tick, npc_id, salt)`. 6 новых файлов
нарушили правило.

Лечение:

1. Везде заменить на `KernelRNG(tick=current_tick, npc_id=np_id,`
`salt="impact")`.
2. `llama_cpp_provider.py:163` — LLM seed должен быть
`hash(state_snapshot + input)` (LLM Pipeline TZ §3.4), не
`random.randint`.
3. `rules_agent.py` d20 — `kernel_rng.randint(1, 20)`.
4. `lint_kernel_rng.py` — расширить, чтобы проверял `rules_agent.py`
и `llama_cpp_provider.py` (сейчас они в whitelist?).

P1-3. Wall-clock в симуляции (7+ сайтов)

Источники: BUG-FB-012 (`world_scheduler.py:32`), BUG-FB-013
(`world_snapshot.py:88-89`), BUG-DLG-006 (`dialogue_queue.py` 7 сайтов),
BUG-FB-037 (`domain/events.py` `EventDT0.create`), BUG-FB-038

(`world_projection_buffer.py`), BUG-FB-039 (`scene_init.py`), BUG-FB-040 (`sqlite_persistence_adapter.py`)

`time.time()` и `datetime.now()` в симуляционном слое. Нарушает §15.1.

Лечение:

1. `dialogue_queue.py` — все `cooldown/rate-limit/enqueued_at` хранить в `tick` (integer), не `time.time()`.
2. `EventDTO.create` — убрать `created_at = time.time()`. Если нужно для лога — писать в CDS, не в DTO.
3. `world_snapshot.py` — `created_at = current_tick` (int) или `game_time_seconds` (float from `scene_state`).
4. `lint_wall_clock.py` — должен падать на этих сайтах. Проверить `whitelist`.

P1-4. `transition_traversal()` FSM имеет 0 call sites

Источник: ADR-TRAV-FSM, `domain_spatial.md` BUG-SPATIAL-005

FSM существует, но никто не вызывает. Все `trav["status"]` пишутся напрямую в `apply_change` и `TraversalExecutionSystem.advance`.

Лечение: см. P0-3. После wired-up — добавить тест `test_traversal_fsm_is_called`.

P1-5. `SpatialFactory` cache возвращает STALE overlay (BUG-SPATIAL-029)

Источник: `spatial_factory.py` (`_cache`)

`set_overlay()` существует, но вызывается только из `__init__`. А* `pathfinding` и `resolve_node` используют устаревшие `reserved_nodes` / `crowd_density` / `risk_zones` / `light_levels` каждый тик.

Лечение:

1. В `SpatialFactory` — добавить `update_overlay(campaign_id, overlay_data)` метод.

2. В `TickOrchestrator._phase_4_movement` (или где применимо) — обновлять overlay перед pathfinding.
 3. Или: вообще не кешировать `SpatialService` — пересоздавать каждый тик (см. лог: Пересобрал `SpatialService` для `city_gate` (`needs_dynamic=True`) × 7 429 — уже происходит, но как warning, а не как design).
-

P1-6. `L1Chronicle.archive_old_events` сломан (БАГ-2 аудита)

Источник: `l1_chronicle.py:240-268`, тест `test_archive_old_events_does_not_crash`

S176 удалил `DELETE FROM l1_chronicle_events` (нарушал §7.12). Теперь таблица растёт бесконечно.

Лечение:

1. Реализовать **archival** (не deletion): переносить старые события в `l1_chronicle_archive` (другая таблица, возможно на диск).
 2. Или: TTL по `tick` (например, держать в памяти последние 10080 тиков = 1 игровая неделя), остальное — в SQLite WAL на диск.
 3. Тест `test_archive_does_not_grow_unbounded`: 100k событий → archive → в active таблице ≤10080.
-

P1-7. `openai_compatible_provider.py` — сломанный импорт (BUG-DLG-041)

Источник: `openai_compatible_provider.py:12`

```
from app.services.llm.llm_provider import LlmProvider # BUG-DLG-041 FIX
```

Файл `llm_provider.py` не существует (правильно: `provider.py`). `ProviderType.OPENAI` (vLLM, LM Studio, Ollama, OpenAI API) полностью неработоспособен.

Лечение: заменить на `from app.services.llm.provider import LlmProvider`. Тест `test_openai_provider_loads`.

P1-8. `pytomorphy3` отсутствует в `requirements.txt` (Д-2)

Источник: аудит v0.5.3.7.2 §3 Д-2

`ImportError` на свежей установке.

Лечение: добавить в `requirements.txt`. Зафиксировать версию.

P1-9. `MUTATIONS.md` `search path` в CDS wrong (НЕСВЯЗНОСТЬ-4)

Источник: `reports/LAST_SESSION.md` ищет `MUTATIONS.md` в корне, а он в `docs/MUTATIONS.md`. CDS сообщает «MUTATIONS.md не найден» — самодиагностика слепа.

Лечение: в `diagnostics/report_renderer.py` — искать в нескольких местах (`./MUTATIONS.md`, `docs/MUTATIONS.md`, `../MUTATIONS.md`). Или захардкодить `docs/MUTATIONS.md`.

P1-10. Двойная правда в угрозах (`_THREAT_TYPES` vs `_EVENT_SEMANTIC_MAP`)

Источник: НЕСВЯЗНОСТЬ-1 аудита, `belief_transition_engine.py`, `event_semantic_tagger.py`

Два независимых источника правды о том, что считать «угрозой». `ObservationLayer` / `BeliefProjector` не реализованы (grep подтверждает — классов с этими именами нет).

Лечение:

1. Реализовать `BeliefProjector` — единый мост между `ObservedFact` и `BeliefState`.
 2. Удалить `_THREAT_TYPES` и `_EVENT_SEMANTIC_MAP` — заменить на динамический `BeliefProjector.classify(fact, context)`.
 3. Тест `test_threat_classification_uses_single_source`.
-

P1-11. Двойная запись в `BeliefState` (НЕСВЯЗНОСТЬ-2)

Источник: `npc_tick_pipeline.py:311` (Phase 5), `memory_manager.py:795` (Phase 3)

Phase 3 (`CoherenceBeliefAggregator`) и Phase 5 (`BeliefTransitionEngine.integrate`) оба модифицируют `BeliefState` . Phase 3 идёт раньше Phase 5. Без merge rule, без lock.

Лечение:

1. Ввести `BeliefMerger` — единый write point. Все belief-updaters складывают deltas в `pending_belief_deltas` , merger применяет их в конце тика с детерминированным приоритетом.
2. Или: оставить только Phase 5 writer; Phase 3 должен только **read** для coherence check, не write.

P1-12. `cannot pickle 'mappingproxy'` в `npc_tick_pipeline.py:206`

Источник: `cds_session_*.log` (57 TICK_CRASH)

```
_npc_dict_for_write = copy.deepcopy(dict(_npc_profile))
```

`_npc_profile` содержит class `__dict__` (mappingproxy). `deepcopy` падает.

Лечение:

1. Заменить на `dataclasses.asdict(_npc_profile)` если это dataclass.
2. Или: `_npc_dict_for_write = {k: v for k, v in vars(_npc_profile).items() if not k.startswith("_")}` .
3. Тест `test_npc_tick_pipeline_does_not_crash_on_mappingproxy` .

P1-13. `create_tick_state() got unexpected kwarg 'l1_chronicle'` (20 TICK_CRASH)

Источник: `tick_orchestrator.py:1224` , `cds_session_*.log`

V8-PSY-1 FIX — half-applied refactor. Caller передаёт `l1_chronicle=...` , функция его не принимает.

Лечение: добавить `l1_chronicle: Optional["L1Chronicle"] = None` в сигнатуру `create_tick_state` (и использовать его). Или убрать `kwargs` из caller.

P1-14. 'TickBuffer' object has no attribute 'campaign_id' (49 раз)

Источник: `cds_session*.log, tick_orchestrator.py:1243`

`_mem_mgr.get_identity_traits(ctx.campaign_id, _nid)` — `ctx` это `TickBuffer`, а не ожидаемый тип.

Лечение: либо добавить `campaign_id` в `TickBuffer`, либо передавать `campaign_id` отдельным аргументом.

P1-15. 'dict' object has no attribute 'stop' в

`llama_cpp_provider.py:155`

Источник: `cds_session*.log` (47 раз)

`params.stop` — но `params` это dict, не объект.

Лечение: `params.get('stop', [])` или обернуть в `GenerationParams` dataclass.

P1-16. cannot import name 'EventType' from 'app.domain.events' (6 TICK_CRASH)

Источник: `phases/input.py:208`

Half-applied refactor: `EventType` убран из `app.domain.events`, но `phases/input.py` всё ещё импортирует.

Лечение: либо вернуть `EventType` в `app.domain.events`, либо убрать импорт.

P1-17. cannot access local variable 'dataclasses' (8 TICK_CRASH)

Источник: `npc_tick_pipeline.py:532`

`UnboundLocalError` — `import dataclasses` не на уровне модуля.

Лечение: добавить `import dataclasses` в начало файла.

P1-18. EventBus DLQ churn (193 события)

Источник: `cds_session_*.log`, `MvpTavernController.on_tick_completed`

```
AttributeError: 'dict' object has no attribute 'all_npcs_raw'  
File "mvp_tavern_controller.py", line 130, in on_tick_completed  
    for npc in ctx.all_npcs_raw:
```

`ctx` это `dict`, а не объект.

Лечение: `for npc in ctx["all_npcs_raw"]:` или обернуть в `TickContext`.

P1-19. Memory event store is None (707 раз)

Источник: `cds_session_*.log`

```
[MEMORY_EVENT] create_memory_event failed for {npc}: 'NoneType'  
object has no attribute 'apply'
```

Memory event store ссылка None. Memory events молча теряются.

Лечение: в `MemoryManager.__init__` — `assert` что `_layered.store` не None. Если инициализация не удалась — кидать `RuntimeError`, не молчать.

P1-20. EventBus validation failed: 'EventBus' object has no attribute '_subscribers' (73 раза)

Источник: `cds_session_*.log` (на каждом старте)

`schema_validator` проверяет `_subscribers`, но `EventBus` был отрефакторен — атрибут убран.

Лечение: либо вернуть `_subscribers`, либо обновить `schema_validator.py` (он сейчас проверяет `ghosts`).

P1-21. 79 туру --strict ошибок в 25/90 файлах

Источник: НАХОДКА-2 аудита v0.5.3.7.2

```
spatial_runtime.py и spatial_service.py — основные источники.  
"None" object is not iterable, tuple | None not indexable, Name  
"SpatialService" is not defined, Statement is unreachable.  
SceneStateManager has no _ensure_spatial_service.
```

Лечение:

1. Запустить туру --strict backend/app/services/spatial/ — исправить top-20 ошибок.
2. SceneStateManager — убрать вызов _ensure_spatial_service (он не существует) или реализовать.
3. После этого — добавить туру в CI как обязательный gate (сейчас туру.ini имеет опечатку [уру] вместо [туру] — конфиг не парсится).

P1-22. туру.ini конфиг сломан (опечатка [уру])

Источник: /туру.ini строка 1

Конфигурация не парсится. Все «строгие» настройки (disallow_untyped_defs = True и т.д.) не применяются.

Лечение: заменить [уру] на [туру]. Запустить туру --strict backend/app/ и убедиться, что правила реально работают.

P1-23. ruff.toml игнорирует все ошибки везде

Источник: /ruff.toml

```
[lint.per-file-ignores]  
"backend/app/models/*" = ["E501", "E402", "F401", "F841"]  
"backend/app/services/*" = ["E501", "E731", "E402", "F821",  
"F841", "F401", "E721"]  
"backend/app/*" = ["E501", "E402", "F401", "F821"]  
...
```

Каждая директория игнорирует F401 (unused import), F841 (unused variable), F821 (undefined name), E501 (line too long). Линтер декоративный.

Лечение:

1. Убрать F821 (undefined name) из всех ignore-списков — это всегда баг.
2. Убрать F401 — unused import засоряет код.
3. Оставить E501 (line too long) — cosmetic.
4. Зафиксировать нарушения.

P1-24. ConfessionParser не реализован (V8-MVP-12)

Источник: ENIGMA_LLM_PIPELINE_TZ_v1.md §1.2, end-screen всегда показывает 0 секретов

NPC confession text никогда не парсится как evidence.

Лечение: реализовать ConfessionParser — extract (npc_id, secret_id, target_npc_id) из LLM response с BNF grammar. Добавить в NpcReplyValidator pipeline.

P1-25. Semantic Cache для LLM не реализован

Источник: аудит v0.5.3.7.2 §4 Phase B

162 LLM-вызова за 45 тиков (1.4 минуты сессии). Cache hit rate = 0% (target ≥35%). При масштабировании кампаний будет узким местом.

Лечение:

1. Реализовать SemanticCache (BGE-small-ru embeddings + FAISS, cosine ≥0.92 = hit).
 2. В ModelRouter.request_for_agent — перед LLM-вызовом проверять cache.
 3. Тест test_semantic_cache_hits_on_repeated_intent .
-

P1-26. Somatic Gate после semantic parsing (BUG-PERC-025)

Источник: `decision_hub.py:402-418`, Causal Contract §4.2.16

DecisionHub парсит semantic action first, потом somatic check. Должно быть наоборот: `shock > 0.7 → skip semantic parsing`.

Лечение: поменять местами блоки в `decision_hub.compute`. Сначала `_check_somatic_gate(state)`, если `shock > 0.7 → return Intent.IDLE` без semantic parsing.

P1-27. BUG-DLG-010: DM-agent читает L2 narrative_cache (L16 violation)

Источник: `dm_phase.py:65-82`, `dm_agent.py:233-236`

DM-agent достаёт `npc_l2_memory_block` и передаёт в LLM prompt. Прямое нарушение Epistemic Boundary.

Лечение: убрать `npc_l2_memory_block` из DM contract. DM должен видеть только manifestations (EmbodiedTraceDTO), не mental state.

P1-28. BUG-FB-002: skip_time не персисит состояние (NPC teleport-back после sleep)

Источник: `game_loop/__init__.py:743-822`

`skip_time` мутирует `_scene` через `_time_skip.skip(...)`, но не вызывает `commit_tick_result` / `unlock_tick`. После sleep NPC телепортируются обратно.

Лечение: в `skip_time` finally-блок —
`scene_manager.commit_tick_result(...)` +
`scene_manager.unlock_tick(...)`.

P1-29. BUG-FB-007: world_diff_applicator пишет life_status не туда

Источник: `world_diff_applicator.py`

Пишет `npc_cache[npc_id]["life_status"] = "DEAD"` в корень NPC, а все читатели смотрят `npc["body_state"]["life_status"]`. Мёртвые NPC считаются живыми в `combat/decision_hub`.

Лечение: писать `npc_cache[npc_id]["body_state"]["life_status"] = "DEAD"`.

P1-30. BUG-FB-001: `stream_turn SSE done` дропает

`world_snapshot`

Источник: `game_loop_bridge.py`, Direct mode

`bridge.read(event.get("world_snapshot"))` — всегда `None`. NPC positions пустые, `player_perception` missing.

Лечение: в `stream_turn` — добавлять `world_snapshot` в финальный `done event`.

P1-31. `ObservationLayer` / `BeliefProjector` не реализованы

Источник: НЕСВЯЗНОСТЬ-1 аудита, грег подтверждает

Лечение: реализовать как часть Phase 2 (Epoch 7). Без них Belief Layer не стабилен.

P1-32. `DIALOGUE_EXEC contract violations` — NPC не могут говорить без STM block

Источник: `sleep_test2.log` (4×), `dialogue_queue.py`

```
[SCHEDULER] Task task-1-blacksmith_orm-dlg failed: NPC
blacksmith_orm cannot speak to
merchant_goran without STM block. Emit approach/greeting intent
first.
```

NPC ↔ NPC диалоги не работают — prerequisite `approach` intent никогда не эмитится.

Лечение:

1. В `DialogueQueue.dequeue_next` — если нет STM block, эмитить `Intent.APPROACH` автоматически.
2. Или: убрать `prerequisite` — позволить dialogue без prior approach (но тогда `RecognitionMemory` должен уметь handle first-contact).

P1-33. `print()` debug statements в production

Источники: `npc_tick_pipeline.py:471`,
`spatial_service.py:533,536,539,605`, `graph_compiler.py:453`,
`game_loop/__init__.py:239,683,685,1097`, 76 occurrences total

3000 строк/сек в stdout для 50 NPC × 60 tps.

Лечение: заменить все на `logger.debug(...)`. Добавить `ruff rule T201` (`print`) как error.

P1-34. Threading model inconsistent

Источник: aggregate search

- `SqlitePersistenceAdapter` имеет `RLock`, `SqliteMemoryStore` — нет
- `MemoryManager` имеет `RLock` для `_identity_cache`, но не для `_dialogue_sessions` / `_pending_dialogue_memories`
- `ModelRouter` объявляет `_worker_lock`, но **никогда не приобретает** его (L165 declared, dead code)
- `ModelPool` использует `asyncio.Lock` (correct for async),
`ProviderManager` — `threading.RLock`

Лечение:

1. Везде, где есть shared mutable state — `threading.RLock` (для sync) или `asyncio.Lock` (для async).
 2. `ModelRouter._worker_lock` — либо использовать, либо удалить.
 3. `MemoryManager` — расширить lock на `_dialogue_sessions` и `_pending_dialogue_memories`.
 4. `SqliteMemoryStore` — добавить `RLock` (или использовать `sqlite3.connect(check_same_thread=True)` и сериализовать через очередь).
-

P1-35. `get_router()` и `get_model_pool()` — race condition на singleton

Источники: `router.py:686-689`, `provider_manager.py:419-423`

```
_router: Optional[ModelRouter] = None
def get_router() -> ModelRouter:
    global _router
    if _router is None:
        _router = ModelRouter()
    return _router
```

Два потока на первом вызове → два `ModelRouter` → два подключения к llama-server.

Лечение: double-checked locking:

```
_lock = threading.Lock()
def get_router() -> ModelRouter:
    global _router
    if _router is None:
        with _lock:
            if _router is None:
                _router = ModelRouter()
    return _router
```

P1-36. BUG-DLG-007/008: Dialogue queue spammed + wall-clock cooldowns

Источники: `task_scheduler.py:121`, `dialogue_queue.py`

`execute_pending()` enqueues ALL pending, dequeues ONE. Rest stays in heap. Cooldowns используют `time.time()`.

Лечение:

1. `execute_pending` — dequeues до rate-limit, не 1.
 2. Cooldowns — на `tick`, не `time.time()`.
 3. `dequeue_next` — возвращать `Optional[DialogueTask]` с explicit reason для None.
-

P1-37. `BREAK_PROGRESS` не учитывает кумулятивность

Источник: `causal_validation.log` "Will state after 15 threats: free"

15 последовательных угроз не ломают will NPC.

Лечение: в `BREAK_DELTA_*` — добавить cumulative bonus: каждая последующая угроза имеет `delta *= 1 + 0.1 * consecutive_count`.

P1-38. `SLEEP_GUARD` срабатывает слишком поздно

Источник: `sleep_test2.log:1615,1620`

`[SLEEP_GUARD] npc=blacksmith_orm routine=sleeping, blocking reactive movement=approach` — блокирует reactive movement, но NPC уже на `exit_west` (после cross-loc intercept).

Лечение: `SLEEP_GUARD` должен проверяться до `_phase_4_movement`, не после.

P1-39. 'NoneType' object has no attribute 'apply' (707 раз в memory pipeline)

Источник: `cds_session_*.log`

Memory event store ссылка None.

Лечение: в `MemoryManager.__init__` — `assert self._layered.store is not None, "MemoryStore init failed"`. Если упало — кидать `RuntimeError`, не молчать.

P1-40. `MockProvider` env mismatch (BUG-FB-021)

Источник: `mock_provider.py:126`

Проверяет `ENIGMA_ENV`, но Settings использует `AIDM_ENVIRONMENT`. Установка одной не меняет другую. `MockProvider` может оказаться в production path.

Лечение: унифицировать env var. Или использовать `settings.environment` (из Settings), не env var напрямую.

Часть 3 — СРЕДНИЕ БАГИ (P2, долг, технические дефекты, не блокируют MVP)

P2-1. God-методы (≥5 штук в hot path)

- `GameLoop._run_pipeline` — 546 строк
- `DecisionHub.compute` — 461 строка
- `LifeEngine` класс — ~1950 строк
- `apply_change` — 236-строчный if/elif
- `NpcTickPipeline.run` — 540 строк
- `StateApplicator._apply_deltas` — 700 строк
- `DmAgent._build_contract` — 570 строк
- `graph_compiler.compile_graph` — 350 строк
- `new_game` — 226 строк с дублированной нумерацией секций

Лечение: разбить на 5–10 методов по 50–100 строк каждый.

Использовать `Extract Method` рефакторинг — но **вручную**, не AI-скриптом.

P2-2. 11+ stub-методов в `tick_orchestrator.py`

Тело методов заменено на `#` Тело метода удалено. Логика перенесена в `phases/X.py`. Файл всё ещё 1759 строк.

Лечение: удалить stub-методы. Если caller'ы вызывают `tick_orchestrator._phase_5_decision(...)`, заменить на прямой вызов `phases.decision.run(ctx, ...)`.

P2-3. 288 `except Exception` в `backend/app`

Лечение: заменить на конкретные исключения. Для мест, где нужно catch-all — переопределить как `except (KeyError, AttributeError, ValueError) as e:` и логировать с уровнем ERROR, не WARNING.

P2-4. 507 `FIX`-тегов, 476 `ADR`-тегов, 120 `BUG`-тегов в комментариях

Кодовая база превратилась в археологический памятник собственным итерациям.

Лечение:

1. Завести `BUGS.md` (этот файл) — перенести туда все теги.
2. В коде оставить только те `ADR-*` теги, которые ссылаются на **актуальные** архитектурные решения (не на «`FIX`:»).
3. Удалить tombstone-комментарии вида `# BUG-CORE-001 FIX: Мёртвый код удалён.` — `git history` хранит информацию.

P2-5. 40+ `fix_*.py` скриптов с `str.replace()`

```
fix_life_engine_mypy.py → _v2 → _v3 → _final .  
fix_tavern_nodes.py → _v2 → _final .
```

Лечение: удалить все `fix_*.py` после применения. Если нужен repeatable рефакторинг — использовать `libcst` или `com2ann`, не `str.replace`.

P2-6. Локальный Windows-путь в исходниках

Источник: `scene_state_manager.py:3` — `# C:\DDD\Codex\VSC_Enigma\Enigma\backend\app\services\scene_state_manager.py`

Лечение: убрать. Если нужен header с путём — использовать относительный путь от `repo root`.

P2-7. Dead code (несколько конкретных случаев)

- `life_engine.py:305-369` — `macro_simulate` с `idle_seconds = 0.0` (55 строк недостижимы)
- `R3_DIRECT_MODE: bool = True` с `else {}` веткой
- `_compute_risk` (66 строк, никогда не вызывается)
- `_get_rel_value` определён дважды с противоречащими шкалами

- `game_loop/__init__.py:2186-2188` — две мёртвые строки после `return state`
- `_removed_social_satiation_modifier` — tombstone-as-method
- `_skip_player_state = False` в `dm_agent.py:380` — всегда False

Лечение: удалить. После удаления — прогнать тесты.

P2-8. Fake metrics (`elapsed_ms = 0.0` formatted as `f"{elapsed_ms:.2f}ms"`)

Источники: `tick_orchestrator.py:328`, `scene_state_manager.py:1507`, `tick_orchestrator.py:510, 525`

Лечение: либо реально измерять (`time.perf_counter()`) — но это wall-clock, запрещён в симуляции!), либо убрать логирование вовсе.

P2-9. `inspect.currentframe()` для debug logging в hot path

Источник: `scene_state_manager.py:437-448`

Тяжёлая introspection на каждой загрузке `scene_state`.

Лечение: убрать. Если нужно знать caller'a — передавать `caller_id: str` параметром.

P2-10. `state.layers = ...` после `return state`

Источник: `game_loop/__init__.py:2186-2188`

Dead statements после return.

Лечение: удалить.

P2-11. `_touch` hack с `len() + 1` вместо монотонного счётчика

Источник: `life_engine.py:375-380`

`float(len(self._last_access)) + 1.0` — ломается при LRU eviction.

Лечение: использовать `itertools.count()` или `self._counter += 1` в `__init__`.

P2-12. `assert` для runtime validation

Источники: `life_engine.py`:573, 2019

`python -O` срезает `assert`'ы.

Лечение: заменить на `if not condition: raise ValueError(...)`.

P2-13. `and/or` short-circuit ternary

Источник: `scene_state_manager.py`:886-888

`"state": obj.get("properties", {}).get("open", True) and "intact" or "closed"` — pre-Python-2.5 idiom.

Лечение: `"intact" if obj.get(...) and ... else "closed"`.

P2-14. `Optional[T]` vs `T | None` смешение

Лечение: унифицировать на `Optional[T]` (если Python 3.9-) или `T | None` (если 3.10+). `target-version = "py311"` в `ruff.toml` — значит можно `T | None`.

P2-15. `print()` в `game_launcher.py`:70 — file handle leak

```
_subprocess_log = open(str(_cds_log_for_subprocess), "a",
encoding="utf-8")
```

Файл открывается, никогда не закрывается. `os._exit(0)` в конце не закрывает Python file handles корректно.

Лечение: использовать `contextlib.ExitStack` или `atexit.register(_subprocess_log.close)`.

P2-16. `_kill_zombies` использует `shell=True` с интерполяцией

Источник: `game_launcher.py:133`

```
subprocess.run(f"netstat -ano | findstr :{port}", shell=True, ...)
```

— security smell, не кроссплатформенно.

Лечение: использовать `psutil.net_connections()` + `psutil.Process(pid).terminate()`. Или `subprocess.run(["netstat", "-ano"], ...)` без shell.

P2-17. `os._exit(0)` после `sys.exit(0)`

Источник: `game_launcher.py:367-369`

Признак того, что корректный выход не работает.

Лечение: разобраться, почему `sys.exit` не срабатывает (незакрытые `subprocess.Popen?` `threading?`). После фикса — убрать `os._exit`.

P2-18. Magic return values (`2 if ok else 0`)

Источник: `scene_state_manager.py:1551`

```
return 2 if ok else 0
```

— почему 2?

Лечение: вернуть `bool` или `enum.CommitResult`.

P2-19. Inline `import` внутри `method bodies`

Источники: `dm_agent.py` (8 раз), `state_applicator.py` (10 раз), `npc_tick_pipeline.py` (8 раз)

Лечение: вынести на уровень модуля. Если circular import — использовать `TYPE_CHECKING` + string annotations.

P2-20. `copy.deepcopy` в `hot loops`

Источники: `npc_tick_pipeline.py` (3× per NPC per tick),
`state_applicator.py` (1× per apply)

Для 50 NPC × 60 tps = 9000 deepcopies/sec.

Лечение:

1. Использовать `dataclasses.replace(state, field=new_value)` вместо `deepcopy`.
2. Или: `pickle.loads(pickle.dumps(state))` — быстрее на 30%.
3. Или: сделать `NPCState` frozen dataclass с structural sharing.

P2-21. `_STOP_TOKENS` содержит пустую строку

Источник: `dm_agent.py`:29-44

`""` в `stop_tokens` → `_has_stop_token("")` всегда True →
`_strip_stop_tokens` обрезает весь ответ до пустого.

Лечение: удалить `""` из списка.

P2-22. `CJK retry path` обходит `DMResponseNormalizer`

Источник: `dm_agent.py`:859-872

Первый attempt нормализует, `retry` — нет. Markdown fences и ChatML теги утекают к пользователю.

Лечение: обернуть `retry` в `DMResponseNormalizer.normalize(raw)` так же, как `first attempt`.

P2-23. `JSONPersistenceAdapter.save_scene_at` — race condition

Источник: `json_persistence_adapter.py`:50-66

Read-modify-write без lock.

Лечение: `threading.Lock` вокруг read-modify-write. Или atomic write pattern (temp + rename).

P2-24. `SQLiteMemoryStore.execute(sql, params)` — публичный API для arbitrary SQL

Источник: `sqlite_store.py:317-330`

SQL injection vector если любой caller передаёт user-influenced SQL.

Лечение: сделать метод `_execute` (private). Внешний API — только типизированные методы (`append`, `recent`, `save_event_memory`).

P2-25. `delete_campaign` использует `LIKE ? с f"%{campaign_id}%"`

Источники: `sqlite_persistence_adapter.py:177-197`,
`sqlite_store.py:308-315`

SQL wildcard injection если `campaign_id` содержит `%` или `_`.

Лечение: escape через `campaign_id.replace("%", "\\%").replace("_", "\\_")` + `ESCAPE '\\'` в SQL.

P2-26. `runtime:{campaign_id}` (no `:loc` suffix) не удаляется в `delete_campaign`

Источник: `sqlite_persistence_adapter.py:265-269, 182-185`

New Game оставляет stale runtime data.

Лечение: в `delete_campaign` добавить `DELETE FROM state_kv WHERE key = ?` (exact match) для `runtime:{campaign_id}`.

P2-27. `assert` в `IPT.py:inv_hp_ssot` — `import os` отсутствует (BUG-A)

Источник: `IPT.py:1038`

`NameError` маскирует HP-SSOT violations.

Лечение: добавить `import os` в начале `inv_hp_ssot`.

P2-28. `inv_dialogue_init` — мёртвый Player → NPC блок (BUG-B)

Источник: `IPT.py:316-358`

Unreachable code после early return at line 314.

Лечение: убрать early return или убрать dead block.

P2-29. `IPT.py:256-257` — `except: pass` нарушает INV-SILENT-FAILURE

Лечение: заменить на `except Exception as e: logger.warning(...)`.

P2-30. `IPT.py:1215-1218` — `except: return passed=True` silent skip

Лечение: возвращать `passed=False` с описанием ошибки.

P2-31. Map topology defects (1 529 wall-crossing)

Источники: `tavern.json` — `entrance→kitchen`, `bar_area→behind_bar`, multiple `city_gate:tent_*` edges blocked by obstacles

Лечение: перерисовать рёбра в `tavern.json` и `city_gate.json` (в `frontend/map_editor/pixels/`). Или добавить `wall_id` к door connections и splitting walls по openings (см. `graph_compiler._split_wall_by_openings`).

P2-32. UTF-8 BOM в `tavern.json` (454× parse errors)

Источник: `graph_compiler.py`

Лечение: `open(path, encoding='utf-8-sig')` вместо `open(path, encoding='utf-8')`.

P2-33. `_local()` возвращает `(0.0, 0.0)` на parse failure

Источник: `spatial_runtime.py:77-83`

Silent fallback to world origin. Combined with `line_of_sight` skipping wall check at (0,0) — entities at origin are always mutually visible.

Лечение: кидать `ValueError("local_position parse failed")` или возвращать `Optional[Tuple[float, float]]`.

P2-34. `get_zone_id` ray-casting bug

Источник: `spatial_service.py:257-267`

`xinters` referenced when `p1y == p2y` (stale from previous iteration).

Лечение: инициализировать `xinters = None` перед `if`, и `if xinters is not None and x <= xinters:`.

P2-35. `_segments_intersect` игнорирует collinear segments

Источник: `graph_compiler.py:466-477`

A* «проходит сквозь углы».

Лечение: добавить обработку collinear case (return True if segments overlap).

P2-36. `traversal_execution_system.advance()` mutates `active_traversals` во время итерации

Источник: `traversal_execution_system.py:30, 111-114`

`RuntimeError: dictionary changed size during iteration` risk.

Лечение: собирать `completed_npcs` (уже делается), но удалять **после** цикла (уже делается). Проблема в `npc_positions[npc_id] = ...` writes во время итерации — вынести в `pending_writes` dict, применять после цикла.

P2-37. `rce.py` fallback присваивает весь DM text как NPC speech

Источник: `rce.py:104-110`

Галлюцинированный диалог в STM.

Лечение: fallback должен возвращать empty list, не `[f"{target_npc_name}: {dm_text}"]`. Если нет quotes — нет speech events.

P2-38. `name_to_id` индексируется каждым словом NPC name

Источник: `rce.py:140-141`

«Купец Горан» индексирует и «купец», и «горан». Если есть «Купец Иван», «купец» → всегда первый NPC.

Лечение: индексировать только по full name (lowercased), не по отдельным словам. Или по фамилии/прозвищу (если есть).

P2-39. `PhenomenologyProjectionService` `elif` chain теряет cues

Источник: `phenomenology_projection_service.py:31-67`

NPC одновременно frozen и shaking получает только `FROZEN` cue.

Лечение: заменить `elif` на `if` для независимых cues.

P2-40. `PerceptionPhysicsEngine` `_compute_confidence` имеет dead code (1.0 - 0.0)

Источник: `perception_physics_engine.py:173-181`

Лечение: убрать или заменить на реальный distortion factor.

P2-41. `PerceptionPhysicsEngine` `angle = 0.0` hardcoded

Источник: `perception_physics_engine.py:46`

Observer видит 360°. Field-of-view cone не применяется.

Лечение: вычислять `angle` из `local_position` (`atan2`).

P2-42. `InferenceEngine` — 5 nested `os.path.dirname`

Источник: `inference_engine.py:32-38`

Fragile — сломается при перемещении файла.

Лечение: `importlib.resources.files("app.config") / "signal_causes.yaml"`.

P2-43. `sound_bleeds_to_adjacent` открывает JSON каждый вызов

Источник: `spatial_runtime.py:385-410`

No caching, called per-NPC per-tick.

Лечение: кешировать в module-level dict с invalidate по mtime.

P2-44. `BehaviorManifestationService` — `logger.info` per NPC per trace

Источник: `behavior_manifestation_service.py:115, 140`

3000 INFO logs/sec for 50 NPC × 60 tps.

Лечение: `logger.debug` или агрегировать в одну INFO per tick.

P2-45. `provider_manager._load_model` unloads active model before checking new provider

Источник: `provider_manager.py:290-291`

Если `ProviderFactory.create` падает — VRAM пуст, fallback'a нет.

Лечение: create-then-swap pattern: сначала создать new provider, потом unload old.

P2-46. `provider_manager._get_model_locked` nested locks

Источник: `provider_manager.py:270-282`

`_switch_semaphore + _pool_lock` nested → deadlock risk.
`asyncio.to_thread` блокирует worker на 30+ секунд, держа оба lock'a.

Лечение: освободить `_switch_semaphore` после acquire `_pool_lock`. Или объединить в один lock.

P2-47. `provider_manager.record_failure` — dead code

Источник: `provider_manager.py:370-375, 322-324`

`record_failure` объявлен, но не вызывается в except блоке
`_load_model`.

Лечение: добавить `self.record_failure(key)` в except.

P2-48. `ModelRouter._abort_generation` ловит все исключения молча

Источник: `router.py:276-284`

Если `pool._active_model` — stale reference, abort no-op, stuck generation продолжается.

Лечение: `except Exception as e: logger.error(...) + retry with backoff.`

P2-49. `ModelRouter.asyncio.run(coro)` в sync method

Источник: `router.py:558-566`

Cross-loop usage crash если `_vram_semaphore` создан на предыдущем loop.

Лечение: создать `_vram_semaphore` lazily inside async context, не в `__init__`.

P2-50. `MemoryManager._pending_dialogue_memories` без lock

Источник: `memory_manager.py:111-127`

Race condition в multi-thread dialogue clearing.

Лечение: `self._dialogue_lock = threading.RLock()` + `with self._dialogue_lock:` вокруг всех mutations of `_dialogue_sessions` и `_pending_dialogue_memories`.

P2-51. `MemoryManager.assess_beliefs` inside `apply()` per NPC per tick

Источник: `memory_manager.py:300-310`

$O(N)$ per tick per NPC. Performance bug.

Лечение: запускать `assess_beliefs` раз в N тиков (например, каждые 10), не каждый тик.

P2-52. `MemoryManager.clear_all_dialogue_sessions` — key parsing assumes 3-part

Источник: `memory_manager.py:132-144`

Если `npc_a` содержит `:`, парсинг ломается.

Лечение: использовать `key.rsplit(":", 2)` или хранить `npc_a` / `npc_b` separately.

P2-53. `state_applicator` swallows `OntologyViolationError`

Источник: `state_applicator.py:110, 259-266`

`apply()` ловит `Exception` (включая `OntologyViolationError`), возвращает original state. `apply_deltas_and_commit` (L1310) — raising. Противоречие.

Лечение: `except OntologyViolationError: raise` перед `except Exception`.

P2-54. state_applicator — duplicate body_state["current_hp"] writes

Источник: state_applicator.py:295-300

Первый write: `min(_max_hp, _new_hp)` . Второй (4 строки ниже):
`int(_new_hp)` — bypasses max_hp clamp.

Лечение: убрать второй write.

P2-55. state_applicator — duplicate DEAD log block

Источник: state_applicator.py:940-947

Двойной `if _life_status == LifeStatus.DEAD: logger.warning(...)` .

Лечение: объединить в один блок.

P2-56. state_applicator — stringly-typed source == "affective_decay"

Источник: state_applicator.py:727-742

Тупо в `source=` строке молча отключает emotion writes.

Лечение: `enum class DeltaSource: AFFECTIVE_DECAY = "affective_decay"` .

P2-57. state_applicator — duplicate ECONOMY handlers

Источник: state_applicator.py:1283-1305

Два блока для `DeltaDomain.ECONOMY` : один для dict payload, один для `EconomicPayload` . Первый съедает второй.

Лечение: убрать первый блок (dict payload). Если есть legacy callers — конвертировать в `EconomicPayload` в adapter'e.

P2-58. `npc_tick_pipeline` mutates `decision.decision.intent_target` post-construction

Источник: `npc_tick_pipeline.py`:562-565

`DecisionResult` объявлен read-only (ADR-TZ10-1).

Лечение: `dataclasses.replace(decision,`
`decision=dataclasses.replace(decision.decision,`
`intent_target="player"))` .

P2-59. `npc_tick_pipeline` monkey-patches `_goal.causal_claims`

Источник: `npc_tick_pipeline.py`:1153-1155

`hasattr(_goal, "causal_claims")` — если frozen dataclass,
`FrozenInstanceError` .

Лечение: добавить `causal_claims: List[CausalClaim] =`
`field(default_factory=list)` в `MacroMovementGoal` dataclass.

P2-60. `ModelPool.__init__` re-entry guard

Источник: `provider_manager.py`:120-148

`getattr(self, "_initialized", False)` — instance is fresh, guard no-op.

Лечение: убрать guard или использовать `__new__` для singleton.

Часть 4 — НИЗКИЕ БАГИ (P3, cosmetic, не влияют на геймплей)

P3-1. 40 ruff style errors (E701, W293, W291, F601)

Лечение: `ruff check --fix backend/` .

P3-2. `version.txt` desync

Лечение: проверить, что `version.txt` совпадает с `ROADMAP.md` текущей версией. CI check.

P3-3. 49 файлов с TODO/FIXME (64 markers)

Лечение: конвертировать в tracked issues в GitHub Issues / TODO board.

P3-4. `event_compiler._resolve_source_xy` — `(0.0, 0.0)` sentinel

Лечение: P2-33.

P3-5. `schema_validator errors` не блокируют запуск (intentional)

Лечение: после стабилизации — сделать blocking.

P3-6. `dict[str, str]` (modern) vs `Optional[dict]` (legacy) — mixed

Лечение: унифицировать.

P3-7. `Tuple` from typing vs `tuple` (PEP 585) — mixed

Лечение: унифицировать на `tuple[...]`.

P3-8. `Path` vs `PathLib` alias for same import

Источник: `game_loop/__init__.py:158`

Лечение: убрать alias.

P3-9. `round(x, 4)` sprinkled redundantly

Лечение: убрать или вынести в `_round4(x)` helper.

P3-10. Russian narrative in docstrings mixing with English ticket IDs

Лечение: стилистический фикс. Документировать convention: docstring на русском, теги на английском, но не вперемешку в одном предложении.

P3-11. `dialogue_queue.dequeue_next` returns `None` for both "empty" and "rate limit"

Лечение: `Optional[DialogueTask]` + `enum.DequeueResult`.

P3-12. `MvpTavernController` hardcoded `tavern_silver_wolf` fallback (BUG-FB-017)

Лечение: parameterize.

P3-13. `routes.py:update_scene_state` принимает `scene_state` от frontend no block-list

Источник: BUG-FB-041

Лечение: allow-list вместо block-list.

P3-14. `routes.py` двойной `/api/api/` prefix на `/api/debug/llm/restart`

Источник: BUG-FB-003

Лечение: убрать дублирование.

P3-15. `BackendContract` missing `set_continuity_mode` / `_base_url`

Источник: BUG-FB-004

Лечение: реализовать.

P3-16. `WorldSnapshotBuilder._empty_snapshot` omits 5 fields

Источник: BUG-FB-005

Лечение: добавить `player_perception`, `avatar_state`, `ambient_phenomenology`, `active_traversals`, `recent_dialogues` со значениями по умолчанию.

P3-17. Save-file schema versioning отсутствует

Лечение: добавить `schema_version: int` в начало каждого save file. Migration logic в `save_format_detector.py`.

P3-18. CI auto lint/pytest on push отсутствует (или не работал из-за `architecture/.github/`)

Лечение: S172 фиксирует это — убедиться, что `.github/workflows/` в repo root, и CI зелёный.

P3-19. `BehaviorManifestationService._safe_get` — defined but unused

Лечение: удалить.

P3-20. `_REGION_MAP` и `_FUNCTION_MAP` как class attrs — нормально, но без type hints

Лечение: добавить `Dict[str, str]`.

Часть 5 — Архитектурные долги (не баги, но риски)

A1. `GameLoop` — god orchestrator

Лезет в приватные атрибуты 6 сервисов:

`memory_manager._relationships`, `scene_manager._persistence`,

`engine._npc_cache`, `engine._temporal._tick_cache`, etc. Архитектура на бумаге слоистая, в коде — звезда с `GameLoop` в центре.

Лечение: вынести в `adapter methods`

(`memory_manager.get_relationships(campaign_id, npc_id)` вместо `memory_manager._relationships[campaign_id][npc_id]`).

A2. 5 незавершённых миграций (из TZ ISPRAVLENIE)

1. **ADR-TZ09-1** (`TickState` / `TickMutation`) — `hub_event` bridge не построен (BUG-CORE-003)
2. **ADR-TRAV-FSM** — 0 call sites (P0-3)
3. **ADR-O-314** (`player_spatial` removed) — writes disabled, reads still active (BUG-SPATIAL-004/032/033)
4. **Dual-Rail (ADR-O-201)** — 135 строк dead code в `tick_orchestrator.execute()` (BUG-CORE-001)
5. **PlayerPerceptionDTO** — два разных класса с одним именем (BUG-PERC-001)

Лечение: закончить каждую миграцию. Завести issue на каждую, с конкретным планом.

A3. Архитектурные линтеры не покрывают свои же инварианты

- `lint_silent_failures.py` ловит `except: pass`, но не `except Exception: log_and_continue` (288 случаев)
- `lint_hp_ssot.py` ловит `state.hp =`, но не `target["hp"] =`
- `lint_kernel_rng.py` ловит `random.`, но `rules_agent.py` и `llama_cpp_provider.py` в `whitelist`?

Лечение: расширить линтеры. После расширения — прогнать и починить все новые нарушения.

A4. CDS search path wrong (MUTATIONS.md)

Лечение: P1-9.

A5. `architecture/.github/` вместо `.github/`

Источник: НАХОДКА-1 аудита. S172 фиксит. Убедиться, что CI в repo root.

A6. `reports/LAST_SESSION.md` — самый свежий 2026-08-06

Лечение: убедиться, что CDS генерит отчёт при каждом запуске. Сейчас, видимо, не генерит (или `game_launcher` падает до экспорта).

A7. Epoch 6 не закрыт

ROADMAP требует закрытия Phase 0 (23 пункта) перед Phase 1. Из них:

- Movement Traversal ❌ (P0-3)
- `REGRESSION-CORE-001` (BUG-CORE-003) partial (P0-11)
- `BREAK_DELTA` rebalance ❌ (P0-6)
- `мыпу --strict` ❌ (P1-21)
- Semantic Cache ❌ (P1-25)
- `canary green` ❌ (несколько P0/P1 блокируют)

Лечение: закрыть все P0 и P1 перед любым горизонтальным расширением.

A8. Сценарий «исследовательский тупик»

Если автор продолжит добавлять Epochs 7–10 (Prophecy, Generations, Society, §18) без закрытия P0 — проект уйдёт в scenario A (см. Часть 0).

Лечение: жёсткий freeze на Epochs 7+ до закрытия всех P0 и 80% P1.

Часть 6 — Карта багов по доменам

Домен	P0	P1	P2	P3	Итого
Core Tick Pipeline (BUG-CORE-*)	4	6	5	2	17
Dialogue/LLM/DM-Agent (BUG-DLG-, BUG-DL-)	3	8	4	1	16
Perception/Combat/Affective (BUG-PERC-*)	1	5	3	0	9
Spatial/Movement/Traversal (BUG-SPATIAL-*)	4	4	6	2	16
Frontend/Persistence (BUG-FB-*)	2	4	5	5	16
LLM Pipeline (BUG-DLG-011/041/043/044)	1	5	3	0	9
Threading/Concurrency	1	3	2	0	6
Config/Tooling (mypy, ruff, CI)	0	3	1	4	8
Design/Balance (BREAK_DELTA, "Нет злодеев")	1	1	0	0	2
Test Infrastructure (IPT, PBT)	0	0	4	1	5
ИТОГО	17	39	33	15	104

Часть 7 — Рекомендованный порядок лечения

Фаза А — Стабилизация запуска (1 неделя, блокирует всё)

1. P0-1 (/api/game/action HTTP 500)
2. P0-2 (TickBuffer self-healing crash)
3. P0-12 (LLM crash-restart churn)
4. P0-14 (scene_changes.jsonl corruption)
5. P1-22 (mypy.ini [ypy] опечатка)
6. P1-23 (ruff.toml ignore-lists)

7. P1-13/14/15/16/17 (5 TICK_CRASH классов)

Gate: `launcher_crash.log` не содержит HTTP 500. CDS session лог не содержит `Traceback` на старте.

Фаза В — Базовая механика (2 недели)

1. P0-3 (Movement Traversal wired-up)
2. P0-4 (social_input_ema save/load)
3. P0-5 (topology_gate для стен без wall_id)
4. P0-7 (Phase 5 decision str.target_id crash)
5. P0-8 (NPC sleep routing)
6. P0-9 (DM «Ничего не произошло»)
7. P0-10 (player_threatens → CombatSubscriber)
8. P0-11 (BUG-CORE-003 hub_event bridge)
9. P0-13 (target_location_id population)
10. P0-15 (INV-TEMPORAL-ISOLATION race)
11. P1-1 (HP Double Truth)
12. P1-2 (random.* violations)
13. P1-3 (wall-clock violations)
14. P1-4 (transition_traversal FSM)
15. P1-28 (skip_time persistence)
16. P1-29 (world_diff_applicator life_status)
17. P1-30 (stream_turn SSE done)
18. P1-32 (DIALOGUE_EXEC prerequisite)

Gate: `sleep_test2.log` завершается ✓ ТЕСТ СНА ПРОЙДЕН.
`causal_validation.log` не содержит «Ничего не произошло».

Фаза С — Дизайн-целостность (1 неделя)

1. P0-6 (BREAK_DELTA rebalance)
2. P1-26 (Somatic Gate ordering)
3. P1-27 (DM-agent L2 leak)
4. P1-37 (BREAK_PROGRESS cumulative)
5. P1-38 (SLEEP_GUARD timing)

Gate: `causal_validation.log` — `identity_integrity` не падает ниже 0.5 после одной атаки + 100 idle тиков.

Фаза D — Архитектурный долг (2 недели)

1. P1-6 (L1Chronicle archival)
2. P1-10/11 (ObservationLayer + BeliefMerger)
3. P1-21 (мыпу --strict 79 errors)
4. P1-31 (ObservationLayer/BeliefProjector)
5. P1-34 (Threading model)
6. P1-35 (singleton race)
7. P1-40 (MockProvider env)
8. P1-24 (ConfessionParser)
9. P1-25 (Semantic Cache)
10. A1 (GameLoop god orchestrator)
11. A2 (5 незавершённых миграций)
12. A3 (расширить линтеры)

Gate: IPT 37/37 green. мыпу --strict 0 errors. ruff 0 errors. CI зелёный.

Фаза E — Cleanup (1 неделя, после D)

1. P2-1 (god-методы)
2. P2-2 (stub-методы в tick_orchestrator)
3. P2-3 (288 except Exception)
4. P2-4 (507 FIX tags — перенести в BUGS.md)
5. P2-5 (40 fix_*.py — удалить)
6. P2-6 (Windows path)
7. P2-7 (dead code)
8. P2-8 (fake metrics)
9. P2-9 (inspect.currentframe)
10. P2-15..17 (game_launcher cleanup)
11. P2-19 (inline imports)
12. P2-20 (deepcopy в hot loops)

Gate: codebase уменьшается на 10–15% по LOC. Никаких `# FIX:` комментариев вне `docs/audits/`.

Часть 8 — Что делать прямо сейчас (следующие 7 дней)

Если бы я был на месте автора, мой план на неделю:

День 1: Остановка кровотечения

- Починить P0-1 и P0-2 (HTTP 500 на старте). Без этого никто не сможет тестировать.
- Починить P1-22 (myru.ini опечатка).
- Прогнать `pytest backend/tests/` — зафиксировать baseline.

День 2: Sleep test green

- Починить P0-7 (Phase 5 str.target_id crash) — 22 TICK_CRASH за сессию.
- Починить P0-8 (NPC sleep routing).
- Запустить `test_sleep_routing.py` — получить `✓ ТЕСТ СНА ПРОЙДЕН`.

День 3: Combat green

- Починить P0-9 (DM «Ничего не произошло») — BUG-DLG-002, BUG-DLG-006, BUG-DLG-018.
- Починить P0-10 (player_threatens → CombatSubscriber).
- Запустить `causal_validation.log` тесты — получить HP < 100 после атаковать трактирщика.

День 4: Save/Load green

- Починить P0-4 (social_input_ema).
- Починить P0-14 (scene_changes.jsonl atomic write).
- Починить P1-28 (skip_time persistence).
- Тест: save → exit → load → NPC positions preserved.

День 5: BUG-CORE-003

- Починить P0-11 (hub_event bridge) — корневая причина «Ничего не произошло».
- Тест: `test_hub_event_reaches_tick_state`.

День 6: LLM stability

- Починить P0-12 (LLM crash-restart churn) — `/abort` endpoint, CJK corruption, VRAM monitor.
- Починить P1-7 (`openai_compatible_provider` import).
- Починить P1-15 (`'dict' object has no attribute 'stop'`).

День 7: Заморозить

- Никаких новых фич.
- Все P0 либо закрыты, либо имеют workaround.
- Commit `V.0.5.3.7.5_stabilization`.
- Написать `RELEASE_NOTES.md` с честным статусом: что работает, что не работает, что дальше.

Часть 9 — Финальное слово

Проект **ENIGMA** — это редкий случай, когда архитектурное видение опережает инженерную дисциплину. Это не плохо — это диагноз.

Автор построил:

- 111 ADR с IMPACT-файлами
- 12 кастомных архитектурных линтеров
- Causal Contract v2.0 с 4 фундаментальными инвариантами
- Dual-Rail equivalence validator с drift-классификацией
- Causal Diagnostic System (CDS) с отчётами при падении
- IPT (Invariant Probe Tests) — 37 инвариантов
- PBT (Property-Based Testing) через Hypothesis
- Self-Healing System (10 уровней, 14 из 15 N-bugs покрываются)

Это уровень инженерии, который большинство коммерческих проектов не достигают.

Но автор также накопил:

- 17 критических багов, из которых 4 блокируют запуск
- 39 высоких багов, ломающих геймплей
- 288 silent except'ов

- 507 `FIX:` tombstone-комментариев
- 40 `fix_*.py` скриптов с `str.replace()`
- God-методы по 540–700 строк
- 5 незавершённых миграций
- Dead code, который никто не удаляет

Шанс, что «всё получится» — есть. Это не безнадежно. Архитектура — фундаментальная, и она работает (Dual-Rail drift C=0, E=0, D=100% — это серьёзный результат). Но чтобы «получилось», нужно сделать то, что автор делает хуже всего: **остановиться и почистить.**

Не добавив Epoch 7. Не написать TZ-20. Не внедрить §18. А просто:

1. Закрывать 17 P0 багов.
2. Закрывать 20 из 39 P1 багов.
3. Удалить 40 `fix_*.py` скриптов.
4. Разбить 5 god-методов на 25 маленьких.
5. Заморозить горизонтальное расширение на 2 месяца.

Если это произойдёт — через 1.5–2 года будет нишевый релиз, который займет пустующее место между Disco Elysium и Dwarf Fortress. Архитектура это позволяет. Вопрос — позволит ли дисциплина.

Удачи.

Часть 10 — Дополнения второго прохода (независимая проверка после первого аудита)

Ниже — находки, которые не попали в первый проход. Часть из них проверена не чтением кода, а прямым исполнением (`pip install`, `pip index versions`), поэтому они не гипотетические.

P0-16. `hypothesis==2.9.2` — версии не существует, `pip install -r requirements.txt` падает целиком

Источник: `backend/requirements.txt:6`

Проверено напрямую: PyPI никогда не публиковал `hypothesis 2.9.2` — после `1.19.0` пакет сразу перешёл на `2.0.0`, версии `2.9.x` в истории

пакета нет вообще (следующая после 2.0.0 — 3.0.0). Команда `pip install -r backend/requirements.txt` из README (шаг 1 Quick Start) **падает на этой строке с `ERROR: No matching distribution found`** ещё до того, как дело доходит до `pygame`, `fastapi` или чего-либо игрового.

Это не косметика: `hypothesis` — не случайная зависимость, а фундамент PBT (Property-Based Testing), который сам аудит (Часть 9) перечисляет как часть инженерной культуры проекта наравне с IPT и CDS. Получается парадокс: слой, который должен доказывать корректность через property-based тесты, не может быть даже установлен по инструкции из README на чистой машине.

Гипотеза о причине: похоже на опечатку или спутанную интерполяцию версии (возможно, имелась в виду 6.92.9 или похожая, либо версия скопирована из другого поля). Требуется уточнения у автора, потому что молча ставить любую версию `hypothesis` — не «лечение», а гадание о том, под какую версию API писались property-тесты.

Лечение:

1. Определить, под какой реальный API `hypothesis` писались PBT-тесты (`grep -rn "from hypothesis" backend/`), затем зафиксировать существующую версию, совместимую с этим кодом.
2. Добавить в CI шаг `pip install --dry-run -r backend/requirements.txt` как быстрый gate до основного прогона — чтобы такие опечатки ловились за 10 секунд, а не после клонирования репозитория новым человеком.
3. Это же — причина, по которой P3-18 («CI не работал») в первом проходе мог наблюдаться не из-за `architecture/.github/`, а из-за банально нерабочего `pip install` на самом первом шаге workflow.

P1-41. CI использует `actions/upload-artifact@v3` — GitHub отключил v3, шаг гарантированно упадёт

Источник: `.github/workflows/test.yml` (шаг Upload test results)

`actions/upload-artifact@v3` официально выведен GitHub из эксплуатации (deprecated, затем отключён) — вызовы этой версии action возвращают ошибку на стороне GitHub, независимо от того, прошли тесты или нет. Даже если P0-16 починить и все тесты позеленеют, CI всё равно завершится с ошибкой на последнем шаге. `actions/checkout@v3` и

`actions/setup-python@v4` пока функциональны, но устарели на 2 мажорных версии.

Лечение: поднять `actions/checkout@v4`, `actions/setup-python@v5`, `actions/upload-artifact@v4`.

P2-61. `pydantic` в `requirements.txt` без версии — единственная незафиксированная зависимость

Источник: `backend/requirements.txt:4`

Все остальные 12 зависимостей записаны точной версией (`==`), кроме `pydantic` (голое имя) и `networkx>=3.1` (нижняя граница). Это единственный по-настоящему «дикий» пин в файле, который иначе очень дисциплинирован.

Проверка кода (`grep -rn "field_validator\|model_config" → 58` совпадений, `grep -rn "@validator\|class Config:" → 0` совпадений) показывает, что кодовая база **последовательно** написана под Pydantic v2 API — это хороший знак, миграционного долга v1 ↔ v2 нет. Но без верхней границы версии сборка не воспроизводима: через полгода `pip install` может подтянуть `pydantic`, который ещё не вышел на момент написания кода, и что-то тихо сломать в валидации 90+ моделей в `backend/app/models/`.

Лечение: `pydantic>=2.5,<3.0` — зафиксировать мажор, но не душить патчи.

P2-62. Второй экземпляр локального пути автора (уже есть похожий в P2-6)

Источник: `backend/requirements.txt:1` — `#`

`C:\DDD\Codex\VSC_Enigma\Enigma\backend\requirements.txt`

Тот же паттерн, что и в `scene_state_manager.py:3` (P2-6), но в другом файле — значит, это не единичная опечатка, а системная привычка (вероятно, автогенерируемый header от инструмента/IDE или AI-агента, вставляющего абсолютный путь). Стоит грепнуть весь репозиторий на `C:\DDD` целиком, а не чинить по одному вхождению.

Лечение: `grep -rln "C:\\\\DDD"` . по всему репо, убрать все вхождения одним PR, добавить pre-commit hook (`.github/` уже существует — логичное место), который блокирует коммит абсолютных Windows-путей.

Р3-21. Кириллические имена файлов с пробелами в `docs/` ломают кросс-платформенную работу с архивом

Источник: `docs/Почти Актуальные TZ/...md`, `docs/audits/209-210-211-212_ПОЛНАЯ АРХИТЕКТУРА...md`, `ADR-0-341_Внедрено.md` и др.

При распаковке репозитория как zip на Linux (`unzip`) несколько файлов с длинными кириллическими именами и пробелами не извлеклись (`File name too long` — суммарная длина пути в UTF-8-байтах превышает лимит файловой системы). Это не баг рантайма, но реальный порог входа: любой инструмент, CI-раннер или коллаборатор на Linux/macOS может получить неполную выгрузку репозитория без явной ошибки в логе выше уровня warning от `unzip` .

Лечение: либо транслитерировать имена файлов в `docs/` (латиница + `_`), либо хотя бы сократить длину — семантика важнее в содержимом файла, чем в 80-символьном кириллическом имени.

Уточнение к Части 0: тестовое покрытие не так однобоко, как звучит афоризм про «1 канарейку»

Источник: `find backend/tests -name "test_*.py" | wc -l` → 172 файла, `find backend/app -name "*.py" | wc -l` → 433 файла.

Формальное соотношение тестовых файлов к прикладным (~40%) заметно лучше, чем создаёт впечатление фраза из Части 0 про «1 канарейку на весь плейтест». Уточнение справедливо только для интеграционных/полных плейтест-тестов (end-to-end от старта до конца сессии) — там дефицит подтверждается. Но модульное покрытие домена, моделей и части сервисов — не заброшено; проблема именно в переходе «юнит проходит» → «интегрированный тик проходит» → «сессия из 40 тиков не крашит», где происходит основной обвал (см. P0-3, P0-7, P0-11).

Часть 11 — Личное мнение аудитора о проекте, его будущем и авторе

Это не продолжение вердикта из Части 0 (там — таблица вероятностей от предыдущего прохода). Это отдельное, субъективное мнение — по прямой просьбе.

О проекте

ENIGMA — не игра в разработке. Это исследовательская программа по формализации причинности в социальной симуляции, которая случайно выбрала форму игры как способ себя проверить. Список артефактов — 111 ADR, 12 архитектурных линтеров, Dual-Rail equivalence validator, эпистемические границы NPC, запрет `random.* / time.time()` в ядре — это не то, что пишут, чтобы выпустить таверну с шестью NPC. Это то, что пишут, чтобы ответить на вопрос «можно ли формально гарантировать, что смоделированный человек не читает чужие мысли и не ломается от одного удара». Таверна — тестовый стенд для этого вопроса, а не продукт.

Отсюда и главное противоречие, которое видно и в первом проходе аудита, и в моей повторной проверке: слой доказательности (IPT, PBT, линтеры, ADR) построен добротнo и местами избыточно строго — а слой, который он должен защищать, ломается на элементарных вещах: `pip install` не проходит (P0-16), NPC не находят свою кровать (P0-8), Movement Traversal не работает вообще ни для одного NPC (P0-3). Разрыв между «мы формально доказали архитектурный инвариант» и «игра не открывает главное меню» — не случайность, а системный паттерн: сложность добавляется быстрее, чем закрывается технический долг предыдущего слоя. Практически весь Часть 9 и Часть 0 первого прохода уже сформулировал это точнее, чем я мог бы, — я лишь подтверждаю диагноз новыми фактами, а не оспариваю его.

О развитии

То, что я нашёл в `reports/LAST_SESSION.md` — секция «ИДЕНТИФИКАЦИЯ АРХИТЕКТОРА» с ролями «Архитектор #1/#2/#3» — говорит о методе работы больше, чем весь остальной аудит. Проект разрабатывается не одним линейным потоком коммитов, а несколькими параллельными AI-агентами (или сессиями), которые координируются через общий

markdown-файл состояния, чтобы не конфликтовать по файлам. Это объясняет одновременно и силу, и слабость кодовой базы:

- **Сила** — отсюда 507 `FIX`-тегов, 476 `ADR`-тегов, 40 `fix_*.py` скриптов, 12 линтеров: это осадок дисциплины, которую пытаются навязать нескольким параллельным потокам работы, чтобы они не разъезжались архитектурно. Без этой дисциплины проект с таким числом параллельных агентов давно бы развалился на несовместимые куски.
- **Слабость** — отсюда же и P1-13/P1-14/P1-16/P1-17/P1-19/P1-20 (сигнатуры не совпадают, атрибуты отсутствуют, импорты не сходятся): классические симптомы того, что несколько потоков меняют смежные контракты не синхронно, а линтеры и ADR ловят это постфактум, а не предотвращают на этапе написания. Автор строит систему сдержек для мультиагентной разработки быстрее, чем сама мультиагентная разработка успевает накопить долг, — и пока проигрывает эту гонку.

Это неортодоксальный, но по-своему логичный способ разработки для одного человека с амбициями на объём кода, который один человек физически не напишет за разумное время. Вопрос не в том, «правильно» это или нет — а в том, что такой режим требует ещё более жёсткой дисциплины по остановке горизонтального роста, чем обычная разработка, потому что каждый новый параллельный поток агентов — это ещё один источник рассинхронизации контрактов. Рекомендация из Части 0 («заморозить Epochs 7-10», «недельный freeze-цикл») в свете этого не пожелание, а необходимое условие: без него параллельные агенты будут физически не успевать выравнивать контракты друг за другом.

О будущем

Я не вижу оснований менять таблицу вероятностей из Части 0 — она честная. Добавлю только одну поправку: находка P0-16 (`hypothesis=2.9.2` не существует) отодвигает точку отсчёта ещё на шаг назад. До сих пор разговор шёл в терминах «игра ломается после старта» — но по факту прямо сейчас **человек, клонирующий репозиторий с нуля по инструкции из README, не пройдёт даже первую команду**. Это не меняет вероятность сценариев В/С из Части 0, но меняет их дедлайн — минимальное условие «сценария В» должно включать нулевой шаг: «сделать так, чтобы чистая машина вообще собралась», раньше, чем «закрыть 17 P0-багов».

Технологическая ставка (сценарий С — статья/фреймворк) кажется мне лично недооценённой в таблице первого прохода. Формализация эпистемических границ NPC, append-only identity chronicle отдельно от ephemeral state, и особенно Dual-Rail equivalence validator с drift-классификацией — это архитектурные решения, которые интересны сами по себе, независимо от того, доедет ли таверна «Серебряный Волк» до релиза. Если бы я расставлял веса, я бы поднял С ближе к 20%, а не 10-15% — при условии, что автор рано или поздно перестанет пытаться делать игру и признает, что уже написал исследовательский фреймворк для каузальной симуляции социальных агентов.

Об авторе (или авторах)

По текстам в README и config/npc — сентиментальная, тонко чувствующая история (Люся, принцип «Нет злодеев», подробные backstory для шести NPC уровня короткой новеллы) — соседствует по соседним каталогам с 12 линтерами уровня production compiler infrastructure. Это редкое сочетание: не «программист, который хочет написать историю», и не «писатель, который выучил Python», а человек, для которого архитектурная строгость — не средство, а часть того же самого эстетического импульса, что и история о Люсе. Формализовать причинность в человеческих отношениях через код — это последовательный проект, а не два разных хобби, случайно оказавшихся в одном репозитории.

Риск, который первый проход уже назвал точно: «визионер, который любит архитектуру больше, чем продукт». Я бы уточнил его иначе — автор любит *доказуемость* больше, чем *завершённость*. 111 ADR — это не тщеславие, это стремление, чтобы каждое решение было прослеживаемо и обратимо. Но доказуемость бесконечно расширяема (всегда можно добавить ещё один инвариант, ещё один слой формализации), а завершённость требует остановиться на «достаточно хорошо» — а это ощущается как проигрыш для человека, который явно способен видеть, как можно сделать лучше. Отсюда и разрыв между 320 файлами docs/ и одной сквозной канарейкой плейтеста.

Это не приговор. Это довольно частый профиль у сильных инженеров с исследовательским складом ума, попавших в проект без внешнего дедлайна. Единственное, что реально нужно — внешний или самоналоженный дедлайн, который сделает «доказуемость» подчинённой по отношению к «работает end-to-end». Судя по TODO.md (сохранение в GitHub, ветка V.0.5.3.7.4_Неплохо_неплохо) и by объёму

проделанной работы — упорства и способности доводить инфраструктурные решения до конца автору хватает с избытком. Не хватает, кажется, готовности признать: движение сквозь шесть дверей таверны без падения — более ценный результат прямо сейчас, чем двенадцатый архитектурный линтер.

Если автор это прочитает и не согласится — это тоже нормально. Это мнение, а не диагноз.